Competitive Programming (CP)

Course Focus: Algorithms, Data Structures, and Problem Solving
Level: Intermediate (Requires C++ or Python proficiency)

Course Description

Competitive programming is like a mental sport. This course is built to sharpen your logic and speed. We aren't just learning "syntax"; we are learning how to look at a complex word problem and translate it into an efficient algorithm that runs in under 1 second. It's the ultimate preparation for technical interviews at companies like Google or Meta.

What You'll Learn

- **Time & Space Complexity** (Big O notation).
- Advanced **Data Structures** (Segment Trees, Graphs, Disjoint Sets).
- Algorithm Paradigms: **Greedy, Dynamic Programming, and Divide & Conquer**.[3]
- Mathematical foundations (Number Theory and Combinatorics).

Weekly Breakdown

- **Week 1: Complexity & STL** – Mastering the Standard Template Library (C++) or Collections (Python).
- **Week 2: Sorting & Searching** – Binary search (on answers), Two pointers, and Sliding window.
- **Week 3: Recursion & Backtracking** – Solving N-Queens, Sudoku solvers, and state-space search.
- **Week 4: Graph Theory Basics** – BFS, DFS, and Topological Sort.[4]
- **Week 5: Shortest Paths & MST** – Dijkstra, Bellman-Ford, and Kruskal's algorithms.
- **Week 6: Dynamic Programming (DP) 101** – Memoization vs. Tabulation. The Knapsack problem.
- **Week 7: Advanced DP** – DP on Trees and Bitmask DP.
- **Week 8: Range Queries & Strings** – Segment Trees, Fenwick Trees, and KMP algorithm.

Assessment

Weekly "Virtual Contests" on platforms like **Codeforces** or **LeetCode**, culminating in a 3-hour final Mock Contest.